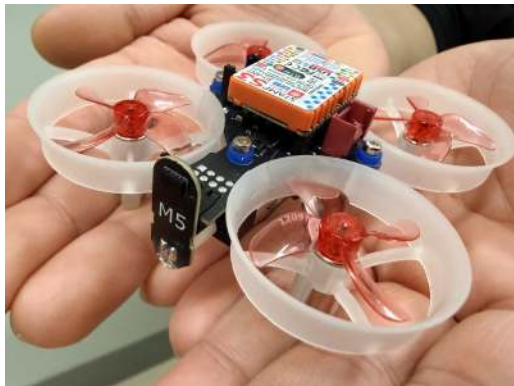


ワークショップカリキュラム / Workshop Curriculum

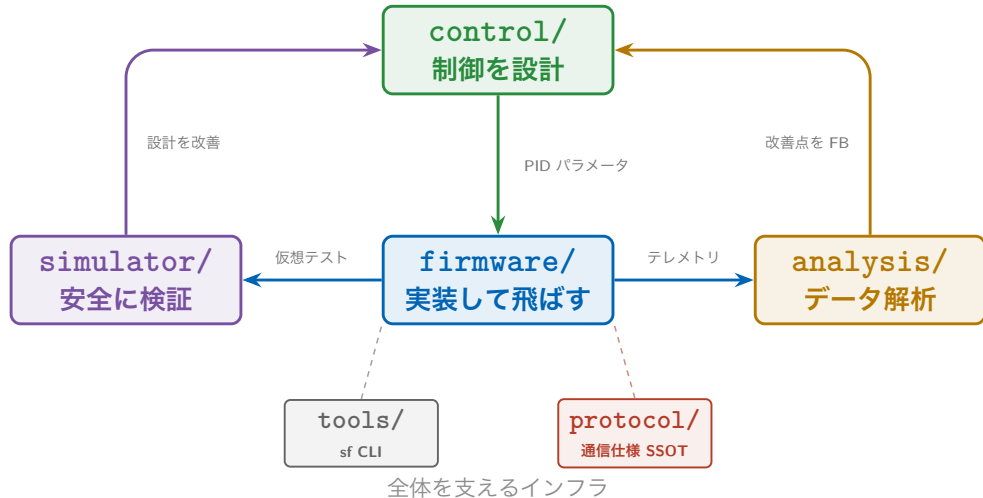
Day	Lesson	テーマ
Day 1	0-2	環境セットアップ + モータ + コントローラ
Day 2	3-5	LED + IMU + 初フライト
Day 3	6-8	モデリング + システム同定 + PID 制御
Day 4	9-12	姿勢推定 + API + 独自 FW + Python SDK
Day 5 AM	13	精密着陸競技会

StampFly ハードウェア / Hardware



項目	仕様
MCU	ESP32-S3 (M5Stamp S3)
IMU	BMI270 (6 軸 400 Hz)
気圧	BMP280
ToF	VL53L3CX
質量	37 g (バッテリー含む)
通信	ESP-NOW + WiFi
バッテリー	LiPo 1S 3.7 V

エコシステム全体像 / Ecosystem Overview



開発ツール sf CLI / Development Tool

カテゴリ	コマンド例	説明
ビルド	<code>sf build / sf flash</code>	ファームウェア開発
診断	<code>sf doctor / sf monitor</code>	デバッグ
ログ	<code>sf log wifi / sf log viz</code>	テレメトリ取得・可視化
シミュレータ	<code>sf sim run</code>	仮想環境で練習
キャリブレーション	<code>sf cal gyro/accel/mag</code>	センサ校正

ワークショップの基本ワークフロー（各レッスン共通）

`sf lesson switch N` → `sf lesson build` → `sf lesson flash`

レッスン切替 → ビルド → 書き込みの 3 ステップで、
コードを書いてすぐに実機で動作確認できます。

Windows 環境構築 (1/2) / Windows Setup

確認コマンド (CMD)	期待される結果	なければ
<code>git --version</code>	git version 2.x	<code>winget install Git.Git</code>
<code>python --version</code>	Python 3.8 以上	<code>winget install Python.Python.3.12</code>

インストール手順 (CMD で実行)

1. リポジトリをクローン
2. `install.bat` を実行 (ESP-IDF + sfcli 自動インストール)
3. `setup_env.bat` で開発環境をアクティベート

```
git clone https://github.com/M5Fly-kanazawa/stampfly_ecosystem
cd stampfly_ecosystem
install.bat
setup_env.bat
```

Windows 環境構築 (2/2) / Driver & Verification

USB シリアルドライバ

CH9102F ドライバをインストール (M5Stack 製品用)

<https://docs.m5stack.com/en/download>

動作確認

`sf doctor` で環境診断 — すべて **OK** になれば完了

macOS / Linux ユーザー

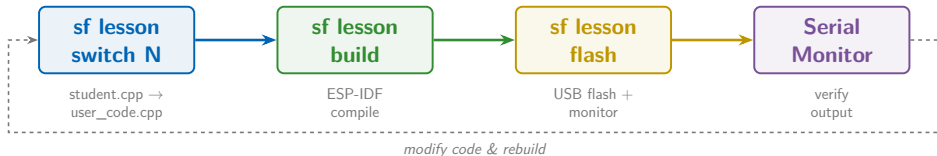
`install.sh` + `setup_env.sh` を使用してください

今日のゴール / Today's Goal

目標

ワークショップファームウェアをビルド・書き込み・動作確認する

- ① `sf lesson build` でビルド
- ② `sf lesson flash` で書き込み
- ③ シリアルモニタで "Hello StampFly!" を確認



sf CLI とは / What is sf CLI?

- StampFly 開発専用のコマンドラインツール
- ESP-IDF のコマンドをシンプルに実行できる

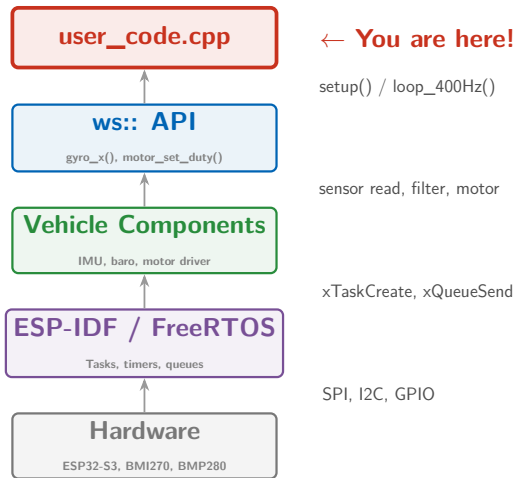
コマンド	説明
<code>sf doctor</code>	環境診断
<code>sf lesson build</code>	ワークショップビルド
<code>sf lesson flash</code>	書き込み+モニタ
<code>sf lesson switch N</code>	レッスン切り替え
<code>sf monitor</code>	シリアルモニタ

ワークショップの構造 / Workshop Structure

学生が実装する関数は2つだけ！

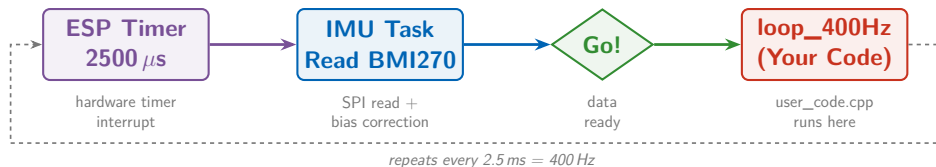
- `setup()` — 起動時に1回呼ばれる
- `loop_400Hz(dt)` — 400Hz (2.5ms 毎) で呼ばれる
- ハードウェアの複雑さは `ws::` 名前空間で隠蔽
- FreeRTOS, SPI/I2C, センサーフュージョンを意識する必要なし

ファームウェアの全体像 / Firmware Architecture



- 皆さんのコードはここ！
`user_code.cpp` を編集するだけ
- `ws::gyro_x()` と書くだけで
SPI 通信→フィルタ→バイアス補正を
全部やってくれる
- 下のレイヤーはワークショップ中
意識する必要なし

400Hz ループの裏側 / Behind the 400Hz Loop



ポイント

`loop_400Hz(dt)` は常に IMU データ更新後に呼ばれる— 学生はタイマー管理不要、ただ関数を書くだけ！

実習: Hello StampFly / Hands-on

user_code.cpp

```
1 #include "workshop_api.hpp"
2 void setup() {
3     ws::print("Hello StampFly!");
4 }
5 void loop_400Hz(float dt) {
6     // Nothing to do in Lesson 0
7 }
```

手順

1. sf lesson switch 0
2. 上のコードを user_code.cpp に書く
3. sf lesson build && sf lesson flash

作業の進め方 / Workflow

ファイル構造

```
firmware/workshop/  
├── lessons/  
│   ├── lesson_00/student.cpp  
│   ├── lesson_01/student.cpp  
│   └── ...  
└── main/  
    └── user_code.cpp ← 編集！
```

- ① `sf lesson switch N`
テンプレートを `user_code.cpp` にコピー
- ② `user_code.cpp` を編集
- ③ `sf lesson build`
- ④ `sf lesson flash`

注意

`student.cpp` はテンプレートなので直接編集しない！
`switch` で何度でもリセットできる

コントローラのセットアップ / Controller Setup

コントローラのビルドと書き込み

- 1 コントローラを USB 接続
- 2 `sf build controller`
- 3 `sf flash controller`

確認

LCD に起動画面が表示されれば OK
Lesson 2 でペアリングして使用する

シミュレータで遊ぶ / Try the Simulator

USB HID モードに切替

- 1 画面タッチでメニューを開く
- 2 Comm: を選択
- 3 USB HID に切替 → 自動再起動

操作	説明
スロットル	上昇
ロール / ピッチ	傾き
ヨー	回転

アクロモード（角速度制御）で飛行

コマンド

```
sf sim run  
sf sim run -w ringworld
```

注意

終了後は Comm: を ESP-NOW に戻すこと（実機飛行用）

チェックポイント / Checkpoint

確認事項

- ☐ `sf doctor` がエラーなしで通る
- ☐ ビルドが成功し "Hello StampFly!" が表示される
- ☐ コントローラのビルド・書き込みが完了した
- ☐ シミュレータが起動し操縦を試した

次のレッスン

Lesson 1: モータ制御 — モータを回してプロペラの動きを確認

Lesson 1

モータ制御

Motor Control

StampFly Workshop